

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА 44

КУРСОВАЯ РАБОТА
ЗАЩИЩЕНА С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

доцент
должность, уч. степень, звание

подпись, дата

Т.Н. Соловьева
инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОМУ ПРОЕКТУ

Бинарные наручные часы

по дисциплине:

Микроконтроллерные системы

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4241

подпись, дата

И.В.Севастьянов
инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. ФОРМИРОВАНИЕ ТЕХНИЧЕСКОГО ЗАДАНИЯ.....	4
1.1. Обзор существующих решений	4
1.1.1. Решение 1 – Коммерческие бинарные часы Tokyoflash	4
1.1.2. Решение 2 – Коммерческие наручные часы с десятичным представлением времени 5	
1.1.3. Решение 3 – Микроконтроллерные бинарные часы	6
1.2. Требования к разрабатываемому устройству	7
2. ПРОЕКТИРОВАНИЕ АППАРАТНОГО ОБЕСПЕЧЕНИЯ	8
2.1. Описание функциональной схемы проекта	8
2.2. Выбор аппаратной реализации.....	9
2.3. Описание принципиальной схемы.....	10
3. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	13
3.1. Описание архитектуры программного обеспечения.....	13
3.1.1. Подсистема хранения состояния.....	13
3.1.2. Подсистема индикации	13
3.1.3. Подсистема программного интерфейса I ² C	14
3.1.4. Подсистема работы с DS1307.....	14
3.1.5. Подсистема преобразований времени	15
3.1.6. Подсистема таймера T0 и системного тика	15
3.1.7. Подсистема обработки кнопок и пользовательского интерфейса	15
3.1.8. Основной цикл программы.....	16
4. ПРОВЕРКА РАБОТОСПОСОБНОСТИ АППАРАТНО-ПРОГРАММНОГО КОМПЛЕКСА.....	17
4.1. Описание проверки устройства требованиям, заявленным в разделе 1.....	17
ЗАКЛЮЧЕНИЕ.....	27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28
ПРИЛОЖЕНИЕ А	29
ПРИЛОЖЕНИЕ Б	30

ВВЕДЕНИЕ

Бинарные часы (также известны как двоичные часы) - часы, использующие для отображения времени двоичную кодировку.

Принцип работы бинарных часов - каждое число (от 1 до 12 часов и от 1 до 60 минут) можно представить в виде суммы двух и более чисел, которые представлены на циферблате. В качестве индикаторов часто используются светящиеся элементы (светодиоды, лампы), где включённому состоянию соответствует цифра 1, а выключенному - 0. Главное отличие бинарных часов - отсутствие циферблата и цифр в привычном понимании, вместо них - яркие неоновые точки. В некоторых моделях цветные светодиодные индикаторы показывают не только время, но ещё и день недели, число, месяц

Актуальность разработки бинарных наручных часов обусловлена популярностью нестандартных и оригинальных гаджетов, применением микроконтроллерных систем в носимой электронике, а также расширением практических навыков в области разработки встроенных систем.

Цель работы - разработка микроконтроллерного устройства «Бинарные наручные часы», отображающего текущее время при помощи светодиодов.

Задачи работы:

- Провести обзор существующих решений и выделить их основные достоинства и недостатки.
- Сформулировать технические требования к разрабатываемому устройству.
- Разработать структурную схему устройства.
- Разработать программное обеспечение для управления наручными часами.
- Провести моделирование и отладку работы устройства.

1. ФОРМИРОВАНИЕ ТЕХНИЧЕСКОГО ЗАДАНИЯ

1.1. Обзор существующих решений

1.1.1. Решение 1 – Коммерческие бинарные часы Tokyoflash

Компания Tokyoflash Japan выпускает серию наручных бинарных часов, где время отображается с помощью светодиодов или ЖК-индикаторов, сгруппированных в замысловатые узоры или ряды [1-3]. Эти устройства позиционируются как аксессуары с уникальным дизайном, сочетающие технологичность и стиль.



Рисунок 1 – Вид часов Logo Binary LCD Watch от компании Tokyoflash Japan

Функциональность устройств, как правило, включает отображение времени (часы, минуты) и даты, выбор между 12- или 24- часовым форматом, а также подсветку дисплея, активируемую нажатием кнопки. Некоторые модели оснащаются анимированной индикацией, которую можно отключать. С точки зрения эксплуатации производитель заявляет о водозащите уровня 3 ATM и использовании стандартной сменной батареи типа CR2025.

Управление часами осуществляется с помощью следующих органов управления:

- одна или две кнопки на корпусе;
- кратковременное нажатие - вывод времени и/или даты;
- длительное нажатие - переход в режим настройки;
- последовательные нажатия - переключение режимов (время, дата, бинарный режим, включение/отключение анимации).

Преимущества: оригинальный дизайн, компактность, готовое изделие.

Недостатки: высокая стоимость

1.1.2. Решение 2 – Коммерческие наручные часы с десятичным представлением времени

Для сравнения с бинарными часами рассмотрим обычные электронные наручные часы, использующие десятичное представление времени. Такие устройства отличаются простотой восприятия, надёжностью и широким набором базовых функций: отображение часов, минут, секунд, даты, будильник, секундомер и подсветка [4].



Рисунок 2 – Наручные часы CASIO F-91W

Основные функции большинства моделей включают: отображение текущего времени (часы, минуты, секунды), календарь (дата, день недели, месяц), будильник, секундомер, подсветку экрана для удобства использования в тёмное время суток. Некоторые современные модели оснащаются дополнительными возможностями – таймером обратного отсчёта, несколькими будильниками, автоматическим переходом на летнее/зимнее время, а также влагозащитой уровня 3 ATM или выше, что позволяет использовать часы при повседневных нагрузках и во время занятий спортом.

Управление часами осуществляется с помощью следующих органов управления:

- MODE - переключение между режимами (часы, будильник, секундомер, настройка);
- START/STOP - запуск и остановка секундомера или таймера;
- RESET - сброс показаний;
- LIGHT - включение под

Преимущества: простота использования, доступность, высокая точность, наличие стандартных функций (дата, будильник, секундомер).

Недостатки: ограниченные возможности расширения и отсутствие оригинальности по сравнению с бинарными часами.

1.1.3. Решение 3 – Микроконтроллерные бинарные часы

Данные бинарные часы представляют собой проект, построенный на микроконтроллере ATmega8. В отличие от традиционных моделей с цифровым дисплеем, такие устройства представляют время в двоичной форме с помощью светодиодов или миниатюрных LED-матриц. Каждый светодиод соответствует определённому разряду числа, что позволяет считывать часы, минуты и секунды по комбинации включённых и выключенных индикаторов. Примером подобного решения являются часы EvILeg-1124 [5].



Рисунок 3 - Микроконтроллерные бинарные часы

Функциональные возможности подобных устройств не ограничиваются только отображением времени. В зависимости от прошивки и схемных решений, бинарные часы могут включать: отображение секунд, даты и дня недели, переключение между 12- и 24- часовым форматами, будильник и таймер обратного отсчёта, автоматическую регулировку яркости светодиодов в зависимости от освещённости, анимацию индикации при смене показаний, энергоэффективный режим ожидания с активацией по нажатию кнопки.

Есть возможность реализации индикации температуры, управление режимами с помощью кнопок или энкодера, а также выбор цветовой схемы отображения. Благодаря гибкости программного обеспечения функционал устройства может быть адаптирован под личные предпочтения пользователя.

Управление часами осуществляется с помощью следующих органов управления:

- «Mode» - вход в режим настройки,
- «Hour+» / «Hour-» / «Min+» / «Min-» - корректировка времени;
- «Clear» - сброс
- «Bip» - режим будильника

В других версиях для навигации по меню могут использоваться энкодеры, обеспечивающие более быстрое и удобное изменение значений

Преимущества: широкий набор функций, оригинальный способ индикации, возможность настройки под пользователя, визуальная привлекательность.

Недостатки: повышенное энергопотребление из-за использования множества светодиодов, необходимость привыкания к способу считывания времени.

1.2. Требования к разрабатываемому устройству

В рамках данной работы был проведен анализ трёх реализацией часов.

Функциональные требования:

1. Отображение текущего времени (часы, минуты, секунды) с помощью светодиодных индикаторов в бинарном коде
2. При отключении основного питания устройство сохраняет работу часов за счёт резервной батареи
3. В качестве управляющего элемента используется микроконтроллер семейства 8051
4. Пользователь должен иметь возможность вручную устанавливать и корректировать время с помощью кнопок управления

2. ПРОЕКТИРОВАНИЕ АППАРАТНОГО ОБЕСПЕЧЕНИЯ

2.1. Описание функциональной схемы проекта

На рисунке 4 представлена функциональная схема подключения компонентов системы.

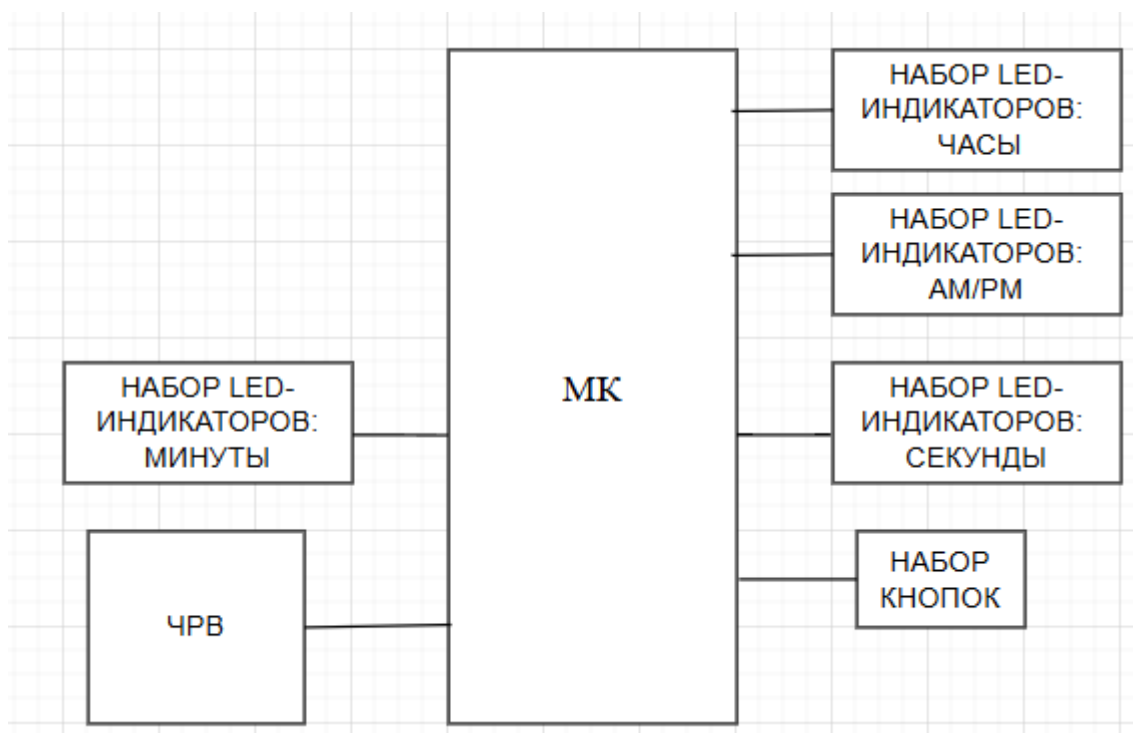


Рисунок 4 – функциональная схема проекта

Функциональная схема отображает взаимодействие основных модулей системы с микроконтроллером (МК). Все элементы подключаются к микроконтроллеру посредством однонаправленных линий связи (одиначные проводники).

Согласно требованиям, система включает следующие функциональные узлы:

1. Микроконтроллер (МК) для выполнения и обработки всех событий и процессов в системе.
2. Часы реального времени (ЧРВ) – модуль, предназначенный для хранения и точного отсчета текущего времени (часы, минуты, секунды). ЧРВ оснащен резервным источником питания (батареей), что обеспечивает сохранение текущего хода времени при отключении основного питания системы.
3. Набор светодиодных индикаторов (Набор LED-индикаторов) – используется для отображения текущего времени:
 - часы – красные светодиоды;
 - минуты – жёлтые светодиоды;
 - секунды – зелёные светодиоды;
 - АМ/РМ-индикатор – два светодиода для 12-часового формата (АМ - красный, РМ - жёлтый).

4. Набор управляющих кнопок (Набор кнопок) – три кнопки (OK, INC, DEC) обеспечивают ручную установку и корректировку времени.

- Кнопка OK вход в режим настройки, переход между параметрами (часы → минуты → секунды) и выход;
- Кнопка INC увеличение значения выбранного параметра: час + 1, минута + 1, секунда +1;
- Кнопка DEC уменьшение значения выбранного параметра: час - 1, минута - 1, секунда - 1.

2.2. Выбор аппаратной реализации

При выборе элементной базы учитывались простота схемы, минимизация стоимости, надёжность и удобство моделирования в Proteus.

Микроконтроллер AT89C52 семейства 8051 выбран как центральный узел. Его ресурсов (8 КБ программной Flash, 256 байт ОЗУ, три 16-битных таймера, частота до 33 МГц) достаточно для статической светодиодной индикации, опроса кнопок и программной реализации обмена с внешними устройствами. В данном проекте взаимодействие с часами реального времени выполняется по I²C, реализованному программно (bit-bang) на линиях портов микроконтроллера.

Часы реального времени DS1307 (64×8) применены для точного хода времени и его сохранения при отключении питания. Модуль работает по I²C, имеет вход VBAT под элемент питания и встроенную энергонезависимую память, что упрощает реализацию требования о резервном питании.

Кварцевый резонатор 32,768 кГц выбран для подключения к DS1307 на выводах X1–X2, поскольку данная микросхема рассчитана на работу именно с “часовым” кварцем такой частоты. Использование 32,768 кГц позволяет получить стабильную базу времени и формировать опорный сигнал 1 Гц с минимальным энергопотреблением, что является стандартным решением для RTC данного класса.

Кнопки BUTTON (SPST Push Button) применяются как простой и надёжный интерфейс пользователя: три штуки (OK, INC, DEC) с подключением одним выводом к входам микроконтроллера, вторым - к земле. Такая схема минимальна по количеству деталей и не требует специальных контроллеров клавиатуры.

Светодиодная индикация выполнена дискретными элементами с фиксированным количеством:

- 4 светодиода для часов (LED-RED);
- 6 светодиодов для минут (LED-YELLOW);
- 6 светодиодов для секунд (LED-GREEN);

- два отдельных индикатора для AM/PM (AM - LED-RED, PM - LED-YELLOW).

Использование отдельных дискретных светодиодов упрощает разводку и повышает читаемость; в Proteus применяются анимированные модели для наглядности.

Резисторная сборка RESPACK-8 (8-канальная с общим выводом) используется для подтяжки линий порта P0 к уровню питания, что необходимо из-за открытого стока микроконтроллера. Это снижает число отдельных элементов и упрощает компоновку. Номинал сборки принят 10 кОм на канал с общим выводом на VCC.

Для ограничения тока светодиодов применены последовательные резисторы 330 Ом в проекте заданы компонентом 10WATT0R1 (для всех каналов индикации). Такое значение обеспечивает безопасный ток порядка 5–10 мА при питании +5 В и соответствует режиму статического включения.

Для линий I²C предусмотрены подтягивающие резисторы порядка 4,7 кОм к +5 В.

Элемент питания CELL (Battery, single-cell) применяется как резервный источник для входа VBAT микросхемы DS1307, что позволяет сохранять ход часов при отсутствии основного питания и полностью удовлетворяет требованию о резервировании.

2.3. Описание принципиальной схемы

Принципиальная схема, составленная в Proteus, приведена в приложении А. Ниже указано подключение микроконтроллера к внешним узлам и особенности включения элементов.

Таблица 1 – Подключение устройств к МК

Устройство	Занимаемые порты	Комментарии
Светодиоды для часов D1–D4 (красные)	P0.0 - P0.3	Биты 1,2,4,8 - веса двоичных разрядов часов (1..12): включённый светодиод добавляет свой вес; сумма = отображаемый час. Катоды на порт, аноды через резисторы к +5 В; «0» на выводе включает светодиод
Светодиоды для AM/PM D17/D18	P0.4 (AM), P0.5 (PM)	Индикаторы формата времени (без весов): горит AM для 00:00 - 11:59, горит PM для 12:00 - 23:59. Катоды на порт, аноды через резисторы к +5 В
Светодиоды для минут D16–D11 (жёлтые)	P1.0 - P1.5	Биты 1,2,4,8,16,32 - веса двоичных разрядов минут (0..59): сумма горящих разрядов = число минут. Катоды на порт, аноды через резисторы к +5 В

Светодиоды для секунд D10–D7 (зелёные)	P2.0 - P2.5	Биты 1,2,4,8,16,32 - веса двоичных разрядов секунд (0..59): сумма горящих разрядов = число секунд. Катоды на порт, аноды через резисторы к +5 В
Кнопка INC	P3.0	Кнопка SPST; второй вывод на GND; активный уровень 0
Кнопка DEC	P3.1	Аналогично кнопке INC
Кнопка ОК	P3.2	Аналогично кнопке INC
Шина I ² C к DS1307 (SCL, SDA)	P1.6 (SCL), P1.7 (SDA)	Две подтяжки по 4,7 кОм к +5 В; линии двунаправленные, открытый сток
RESPACK-8	P0.0 - P0.7	Резисторная сборка подтягивает все линии порта P0 к +5 В; общий вывод сборки - на VCC

Особенности подключения:

Индикация выполнена в статическом режиме: каждый светодиод подключён индивидуально. Аноды светодиодов идут к +5 В через токоограничивающие резисторы, катоды - на выводы микроконтроллера. Нулевой уровень на выводе включает соответствующий светодиод.

Порт P0 микроконтроллера имеет «открытый сток» и не удерживает высокий уровень без внешней подтяжки, поэтому применена сборка RESPACK-8: общий вывод сборки подключён к VCC, выводы 2–9 - к линиям P0.0...P0.7.

Узел DS1307 подключён к кварцу 32,768 кГц (выводы X1–X2). На вход VBAT подана одноэлементная батарея CELL с напряжением 3,0 В; минус батареи - на общую

землю. При отсутствии основного питания +5 В DS1307 продолжает отсчёт времени от VBAT; обмен по I²C возобновляется после восстановления питания.

3. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1. Описание архитектуры программного обеспечения

Программное обеспечение реализует электронные часы на микроконтроллере семейства 8051 (AT89C52) с аппаратным таймером T0, двоичной светодиодной индикацией времени и внешними часами реального времени DS1307 по интерфейсу I²C, реализованному программно. В программе используется 27 функций, разделённых на подсистемы по назначению:

- подсистема индикации - 5 функций;
- подсистема кнопок и пользовательского интерфейса - 1 функция;
- подсистема таймера и системного тика - 2 функции;
- подсистема низкоуровневого I²C - 10 функций;
- подсистема работы с DS1307 - 5 функций;
- подсистема преобразований и форматов времени - 3 функции;
- функции инициализации и основной цикл - 1 функция.

Такое разбиение отделяет аппаратно-зависимые части (таймер, порты, программный I²C) от логики пользовательского управления и индикации времени.

3.1.1. Подсистема хранения состояния

Текущее время и состояние интерфейса хранится в глобальных переменных: hh, mm, ss - текущее время в 24-часовом внутреннем формате;

- mode_24h - режим 12/24 часов;
- have_rtc - результат обмена с DS1307;
- ui - состояние пользовательского интерфейса (ST_SHOW, ST_SET_H, ST_SET_M, ST_SET_S);
- ms, tick_1s, prev_inc, prev_dec, prev_ok, ok_hold_ms, ok_long_fired - переменные системного тика и обработки кнопок.

Данные переменные используются всеми остальными подсистемами и обеспечивают единое состояние устройства.

3.1.2. Подсистема индикации

Подсистема управляет светодиодами, подключёнными к портам микроконтроллера. Состав функций:

1. set_hour_leds()
2. set_min_leds()
3. set_sec_leds()
4. set_ampm()
5. refresh_display()

Логика работы: `refresh_display()` берёт значения `hh`, `mm`, `ss` и флаг `mode_24h`. Для часовой группы формируется значение, выводимое на 4-битные светодиоды часов, затем вызываются `set_hour_leds()`, `set_min_leds()`, `set_sec_leds()`. Состояние индикаторов АМ/РМ задаётся вызовом `set_ampm()` по текущему часу и выбранному режиму формата времени. Таким образом, `refresh_display()` является единой точкой обновления всей индикации устройства.

Как вызывается: `refresh_display()` вызывается в основном цикле после обновления данных времени и после изменения времени в режиме настройки.

3.1.3. Подсистема программного интерфейса I²C

Интерфейс I²C реализован на линиях P1.6 (SCL) и P1.7 (SDA). Состав функций:

1. `sda_release()`
2. `sda_low()`
3. `scl_release()`
4. `scl_low()`
5. `i2c_delay()`
6. `i2c_start()`
7. `i2c_stop()`
8. `i2c_write()`
9. `i2c_read()`
10. `i2c_ack_cycle()`

Логика работы: функции управления линиями формируют физические уровни SDA/SCL. `i2c_start()` и `i2c_stop()` создают стандартные условия начала и окончания сеанса. `i2c_write()` последовательно передаёт 8 бит и считывает АСК от ведомого устройства. `i2c_read()` принимает 8 бит и выставляет АСК/НАСК со стороны мастера. Временные интервалы задаются `i2c_delay()`.

Как вызывается: функции I²C вызываются из подсистемы DS1307 при чтении и записи регистров часов реального времени.

3.1.4. Подсистема работы с DS1307

Подсистема RTC использует низкоуровневый программный I²C. Состав функций:

1. `ds1307_write_reg()`
2. `ds1307_read_reg()`
3. `ds1307_start_osc_if_needed()`
4. `ds1307_read_time()`
5. `ds1307_write_time()`

Логика работы: `ds1307_read_reg()` и `ds1307_write_reg()` выполняют доступ к конкретному регистру DS1307. `ds1307_start_osc_if_needed()` контролирует запуск осциллятора через бит CH в регистре секунд. `ds1307_read_time()` считывает секунды, минуты и часы, переводит значения из BCD в двоичный вид и формирует значения hh, mm, ss во внутреннем 24-часовом формате. `ds1307_write_time()` преобразует hh, mm, ss в BCD и записывает их в регистры DS1307 при завершении пользовательской корректировки времени.

Как вызывается: `ds1307_read_time()` вызывается в основном цикле по флагу `tick_1s`. `ds1307_write_time()` вызывается при выходе пользователя из режима настройки обратно в режим отображения времени.

3.1.5. Подсистема преобразований времени

Состав функций:

1. `bcd2bin()`
2. `bin2bcd()`
3. `to12()`

Логика работы: `bcd2bin()` и `bin2bcd()` обеспечивают корректный обмен с DS1307, который хранит время в BCD. `to12()` преобразует hh из диапазона 0...23 к формату 1...12 для формирования кода часовой группы и корректной работы индикаторов AM/PM.

Как вызываются: `bcd2bin()` используется внутри `ds1307_read_time()`. `bin2bcd()` используется внутри `ds1307_write_time()`. `to12()` используется внутри `refresh_display()`.

3.1.6. Подсистема таймера T0 и системного тика

Состав функций:

1. `timer0_init()`
2. `timer0_isr()`

Логика работы: `timer0_init()` конфигурирует T0 на периодический прерыватель 1 мс. `timer0_isr()` увеличивает счётчик ms и выставляет флаг `tick_1s` при достижении 1000 мс. В этом же обработчике измеряется длительность удержания кнопки ОК в `ok_hold_ms`, что обеспечивает распознавание короткого и длинного нажатий на уровне системного тика.

Как вызывается: `timer0_isr()` вызывается аппаратно системой прерываний. `timer0_init()` вызывается из `main()` при инициализации устройства.

3.1.7. Подсистема обработки кнопок и пользовательского интерфейса

Состав функций:

1. `process_keys()`

Логика работы: process_keys() опрашивает входы кнопок OK/INC/DEC и использует состояние ui для выбора поведения:

- короткое нажатие OK переключает состояния интерфейса по циклу ST_SHOW → ST_SET_H → ST_SET_M → ST_SET_S → ST_SHOW;
- длинное нажатие OK изменяет mode_24h;
- INC/DEC изменяют hh, mm или ss в зависимости от активного состояния настройки.

Как вызывается: process_keys() вызывается в каждой итерации основного цикла.

3.1.8. Основной цикл программы

Состав функций:

1. main()

Логика работы: main() выполняет настройку портов ввода-вывода, запускает таймер через timer0_init(), иницирует работу с DS1307 и устанавливает флаг have_rtc. После инициализации выполняется бесконечный цикл, в котором реализованы три ключевые действия:

1. обработка события tick_1s и вызов ds1307_read_time();
2. обработка пользовательского ввода вызовом process_keys();
3. обновление индикации вызовом refresh_display().

При завершении режима настройки выполняется запись новых значений времени в RTC вызовом ds1307_write_time().

4. ПРОВЕРКА РАБОТОСПОСОБНОСТИ АППАРАТНО-ПРОГРАММНОГО КОМПЛЕКСА

4.1. Описание проверки устройства требованиям, заявленным в разделе 1.

Проверка работоспособности разработанного устройства «Бинарные наручные часы» выполнялась методом моделирования в среде Proteus. Верификация проводилась в соответствии с функциональными требованиями, сформулированными в п.1.2.

Требование 1. Отображение текущего времени (часы, минуты, секунды) с помощью светодиодных индикаторов в бинарном коде

В среде Proteus была запущена модель с микроконтроллером AT89C52 и модулем часов реального времени DS1307. После подачи питания выполняется чтение времени по программной шине I²C. В журнале моделирования (Simulation Log) фиксируется текущее время RTC, например: **18:43:51**.

Так как в разработанной версии используется **12-часовой формат**, при времени 18:43:50 устройство отображает **6:43:51 PM**, поэтому активируется светодиод **PM (D18)**, а светодиод **AM** выключен (рис. 5).

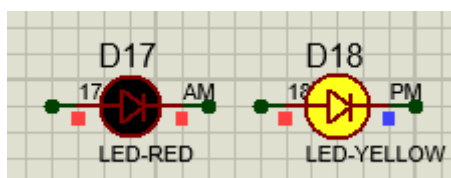


Рисунок 5 – Индикация часового формата

Индикация часов

Название светодиода	Вес
D2	1
D4	2
D3	4
D1	8

Часовой разряд формируется четырьмя красными светодиодами с весами 1, 2, 4, 8. Для отображения значения **6** должны быть включены разряды **2 и 4**. На схеме это соответствует горящим красным светодиодам часовой группы (рис. 6). Сумма весов 2+4 даёт значение 6, что соответствует 18 часам в формате 12h с признаком PM.

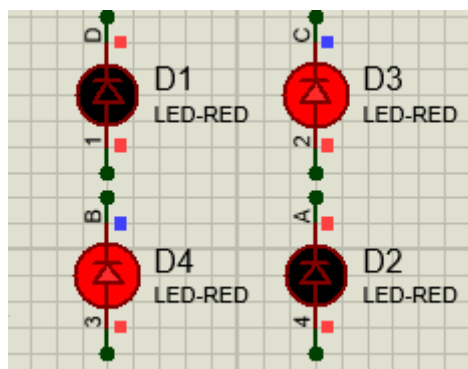


Рисунок 6 - Индикация часов

Индикация минут

Название светодиода	Вес
D13	1
D12	2
D11	4
D14	8
D15	16
D16	32

Минутный разряд формируется шестью жёлтыми светодиодами с весами 1, 2, 4, 8, 16, 32. Значение **43** представляется как сумма $32 + 8 + 2 + 1$, поэтому должны быть активны соответствующие четыре разряда минутной группы. Наблюдаемая комбинация включённых жёлтых светодиодов на схеме соответствует этому разложению (рис. 7).

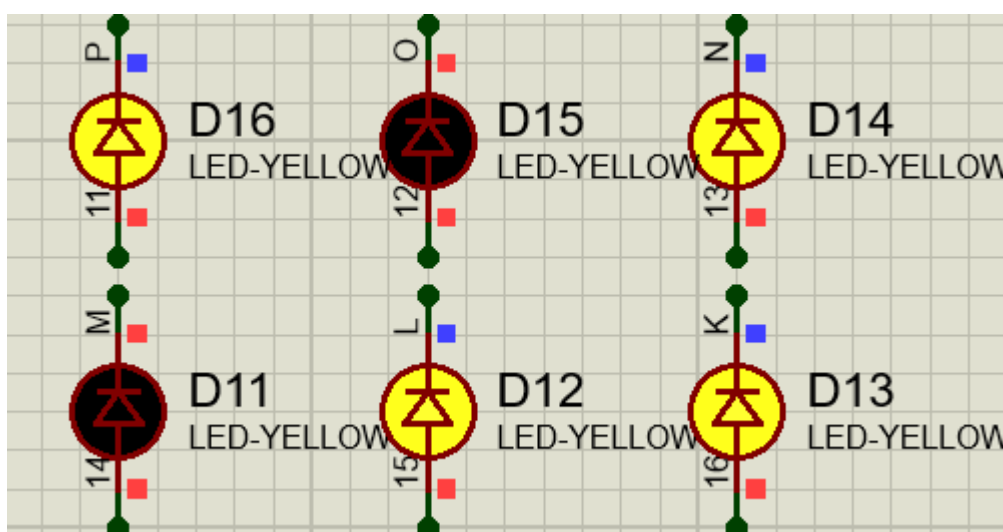


Рисунок 7 - Индикация минут

Индикация секунд

Название светодиода	Вес
D7	1
D6	2
D5	4
D8	8
D9	16
D10	32

Секундный разряд формируется шестью зелёными светодиодами с весами 1, 2, 4, 8, 16 и 32. Значение **51** представляется как сумма $32 + 16 + 2 + 1$, поэтому должны быть включены светодиоды с весами 32, 16, 2, 1. Наблюдаемая комбинация включённых индикаторов соответствует данному разложению (рис. 8).

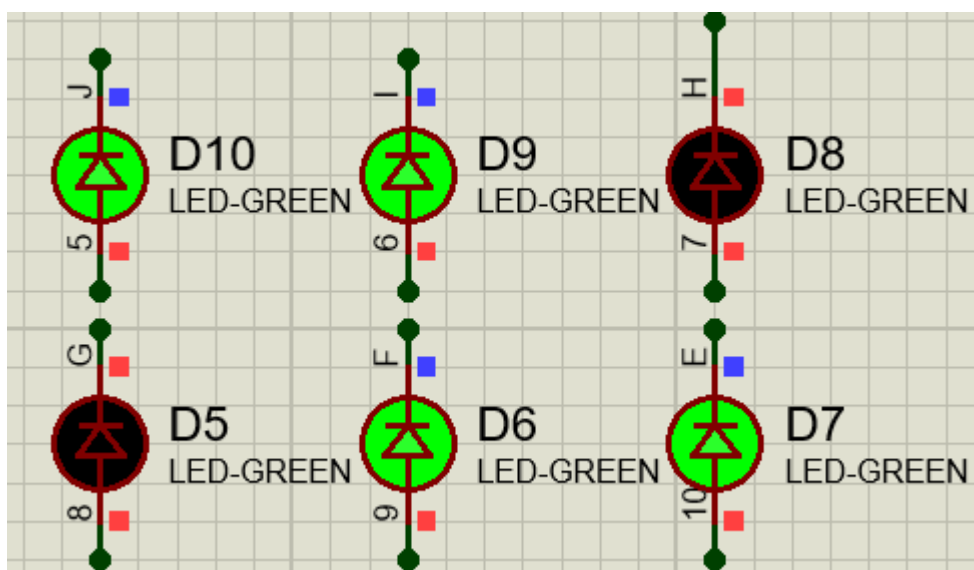


Рисунок 8 - Индикация секунд

Таким образом, при моделировании подтверждено корректное отображение времени в бинарном виде: часовая группа отображает 6 в сочетании с активным индикатором РМ, минутная и секундная группы формируют комбинации, соответствующие 43 и 51 соответственно.

Требование 2. При отключении основного питания устройство сохраняет работу часов за счёт резервной батареи

В ходе моделирования выполнялась имитация отключения основного питания +5 В при подключённом источнике резервного питания на выводе VBAT микросхемы DS1307. После паузы питание было восстановлено. Контроллер корректно считывал текущее время, которое продолжало увеличиваться на длительность паузы, без сброса к начальному значению.

Следует отметить, что в среде Proteus модель DS1307 может сохранять ход времени при отключении VCC независимо от наличия внешнего источника VBAT, что связано с особенностями программной модели. В реальном устройстве сохранение времени обеспечивается резервной батареей, подключённой к выводу VBAT.

Пример проверки: после запуска симуляции устройство отображает время 8:20 (секунды не учитывались, так как в момент подготовки скриншотов время продолжало идти).

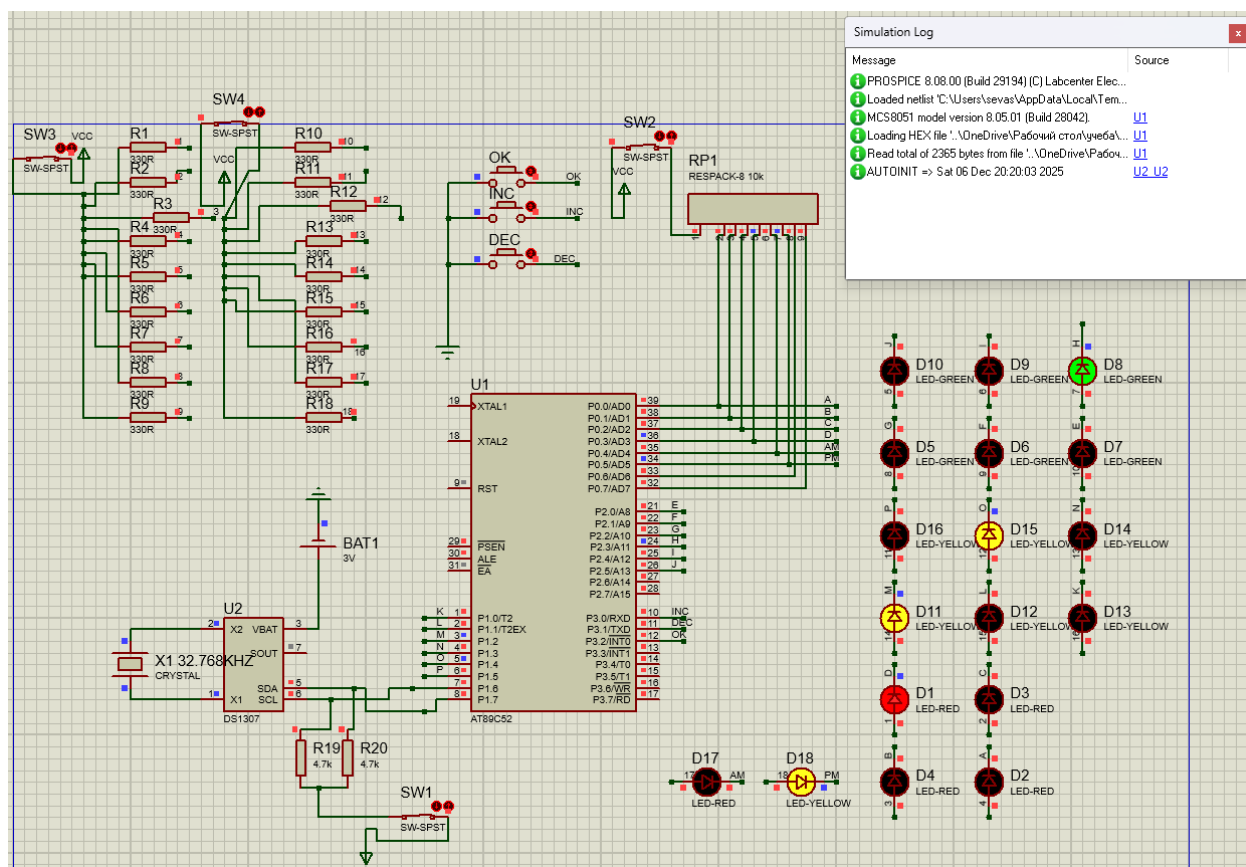


Рисунок 9 – Включенная программа

Затем основное питание отключается с помощью переключателя SW-SPST.

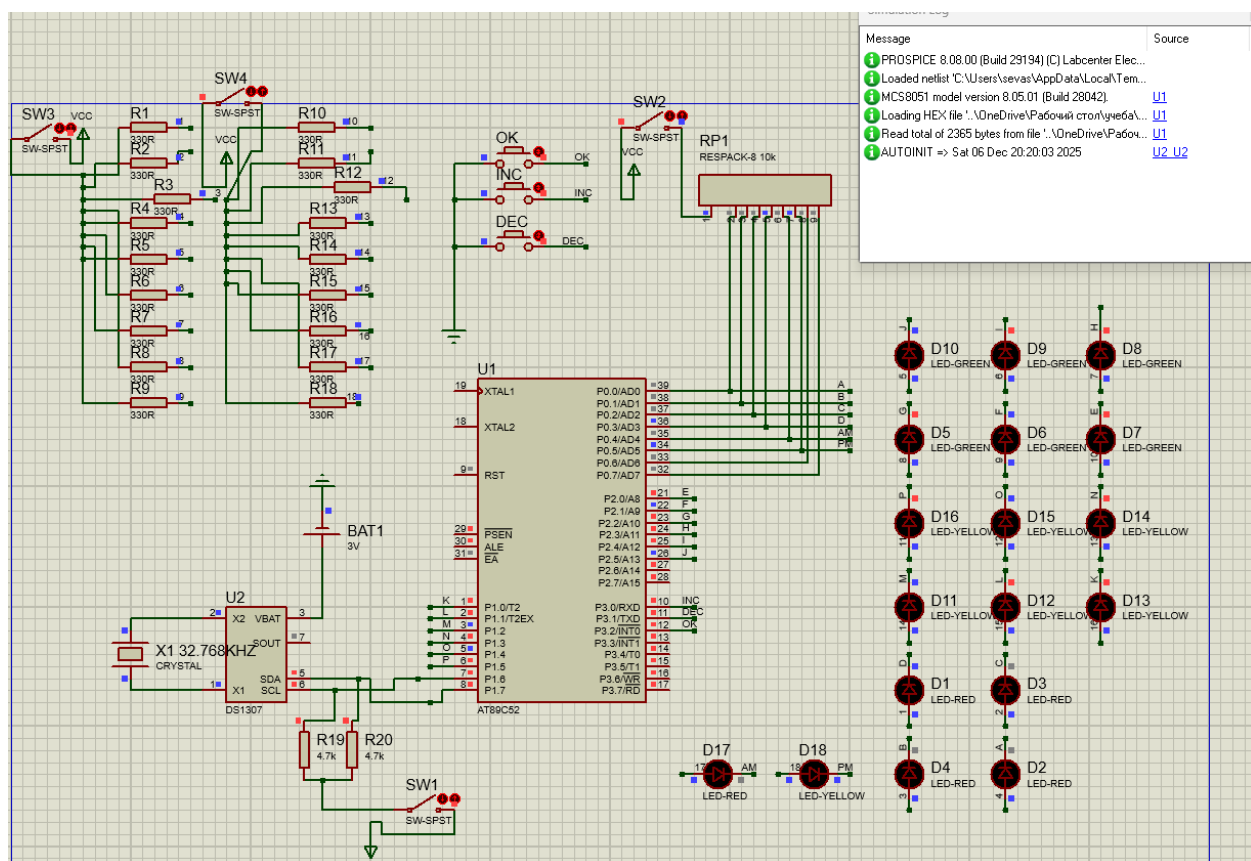


Рисунок 10 – Отключение основного питания

По прошествии некоторого времени питание восстанавливается. Устройство корректно считывает время из DS1307 и отображает актуальное значение 8:23, что подтверждает сохранение хода времени при отсутствии основного питания.

Showing local results: 15

Device	Library	Description
AT89C1051	MCS8051	8-bit microcontroller with 1K code flash and 64-bit iram
AT89C2051	MCS8051	8-bit microcontroller with 2K code flash and 128-bit iram
AT89C4051	MCS8051	8-bit microcontroller with 4K code flash and 128-bit iram
AT89C51	MCS8051	8051 Microcontroller (4kB code, 33MHz, 2x16-bit Timers, UART)
AT89C51.BUS	MCS8051	8051 Microcontroller (4kB code, 33MHz, 2x16-bit Timers, UART)
AT89C51RB2	MCS8051	8051 Microcontroller (16kB code, 48MHz, Watchdog Timer, 3x16-bit Timers, UART)
AT89C51RB2.BUS	MCS8051	8051 Microcontroller (16kB code, 48MHz, Watchdog Timer, 3x16-bit Timers, UART)
AT89C51RC2	MCS8051	8051 Microcontroller (32kB code, 48MHz, Watchdog Timer, 3x16-bit Timers, UART)
AT89C51RC2.BUS	MCS8051	8051 Microcontroller (32kB code, 48MHz, Watchdog Timer, 3x16-bit Timers, UART)
AT89C51RD2	MCS8051	8051 Microcontroller (64kB code, 40MHz, Watchdog Timer, 3x16-bit Timers, UART)
AT89C51RD2.BUS	MCS8051	8051 Microcontroller (64kB code, 40MHz, Watchdog Timer, 3x16-bit Timers, UART)
AT89C52	MCS8051	8052 Microcontroller (8kB code, 256B data, 33MHz, 3x16-bit Timers)
AT89C52.BUS	MCS8051	8052 Microcontroller (8kB code, 256B data, 33MHz, 3x16-bit Timers)
AT89C55	MCS8051	8032 Microcontroller (20kB code, 256B data, 33MHz, 3x16-bit Timers)
AT89C55.BUS	MCS8051	8032 Microcontroller (20kB code, 256B data, 33MHz, 3x16-bit Timers)

Preview

VSM DLL Model [MCS8051.DLL]

Pin	Signal	Pin	Signal
19	XTAL1	39	P0.0/AD0
18	XTAL2	38	P0.1/AD1
9	RST	37	P0.2/AD2
29	PSEN	36	P0.3/AD3
30	ALE	35	P0.4/AD4
31	EA	34	P0.5/AD5
1	P1.0/T2	33	P0.6/AD6
2	P1.1/T2EX	32	P0.7/AD7
3	P1.2	21	P2.0/A8
4	P1.3	22	P2.1/A9
5	P1.4	23	P2.2/A10
6	P1.5	24	P2.3/A11
7	P1.6	25	P2.4/A12
8	P1.7	26	P2.5/A13
		27	P2.6/A14
		28	P2.7/A15
		10	P3.0/RXD
		11	P3.1/TXD
		12	P3.2/INT0
		13	P3.3/INT1
		14	P3.4/T0
		15	P3.5/T1
		16	P3.6/WR
		17	P3.7/RD

PCB Preview

DIL-40

Found more results at 'Component Search Engine', press to view

OK Отмена

Рисунок 12 – Вид модели AT89C52 семейства MCS8051 в библиотеке

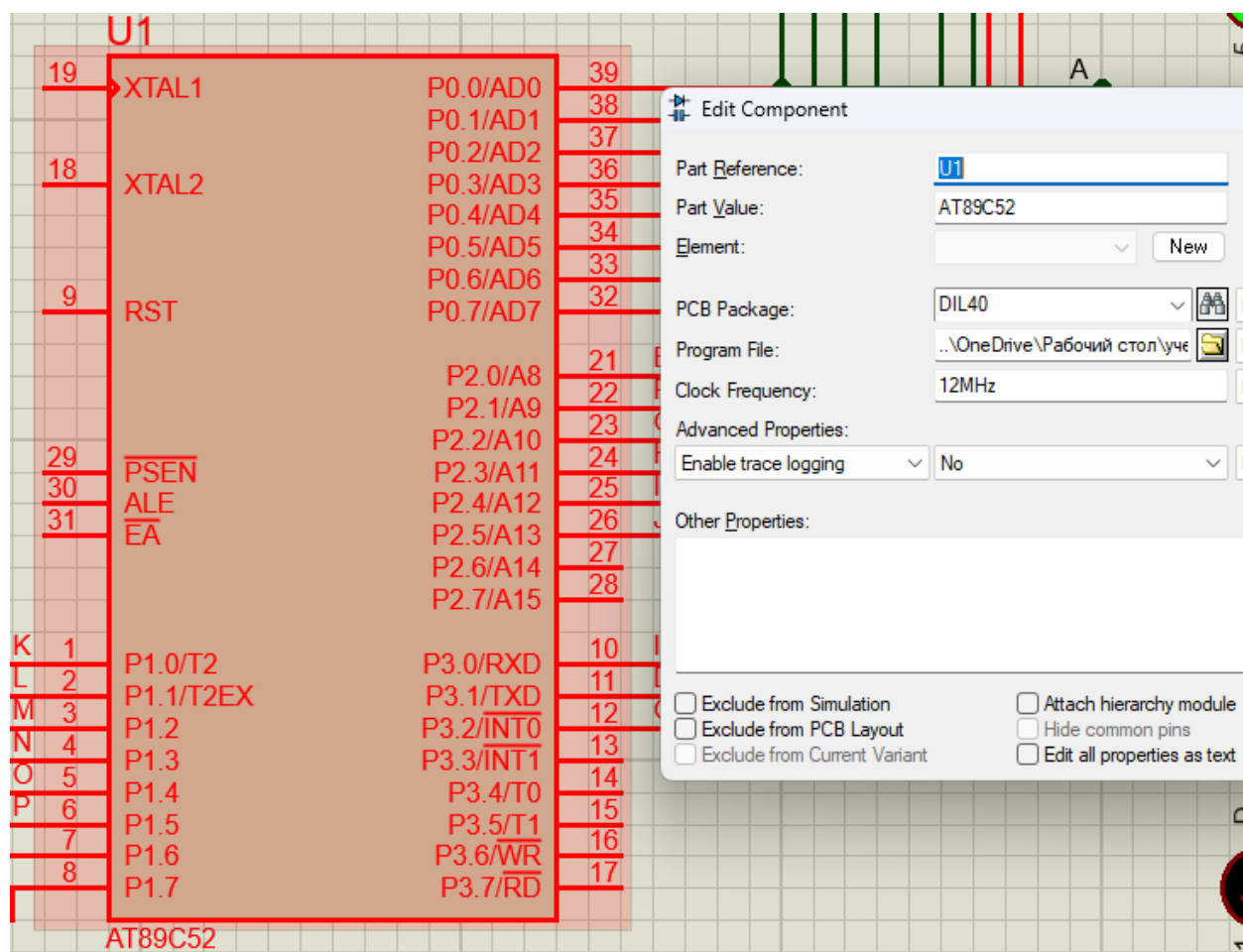


Рисунок 13 – Вид модели AT89C52 семейства MCS8051 в проекте

Требование 4. Пользователь должен иметь возможность вручную устанавливать и корректировать время с помощью кнопок управления

После запуска программы пользователь может управлять устройством с помощью кнопок **OK**, **INC** и **DEC**.



Рисунок 14 – Внешний вид кнопки OK

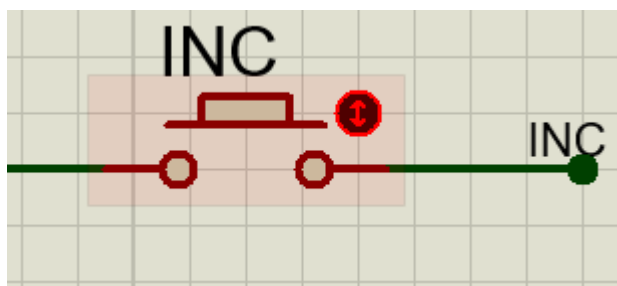


Рисунок 15 – Внешний вид кнопки INC



Рисунок 16 – Внешний вид кнопки DEC

Для перехода в режим настройки времени используется кнопка **ОК**. После нажатия кнопки **ОК** устройство переходит в режим редактирования часов, и автоматический ход времени приостанавливается. Последовательные нажатия **ОК** обеспечивают переход между редактируемыми параметрами времени:

1. первое нажатие - режим редактирования **часов**;
2. второе нажатие - режим редактирования **минут**;
3. третье нажатие - режим редактирования **секунд**;
4. четвёртое нажатие **выход** из режима редактирования и возврат в режим отображения.

Изменение выбранного параметра осуществляется кнопками:

- **INC** - увеличение значения на 1;
- **DEC** - уменьшение значения на 1.

После выхода из режима редактирования часы продолжают работу с установленными пользователем параметрами. При завершении моделирования и повторном запуске встроенные программные значения времени не сохраняются, и текущее время снова считывается из микросхемы **DS1307**.

Рассмотрим настройку на примере редактирования **часов**. При первом нажатии кнопки **ОК** устройство уже переходит в режим редактирования часов, при времени 18:43:50 устройство отображает **6:43:51 PM**:

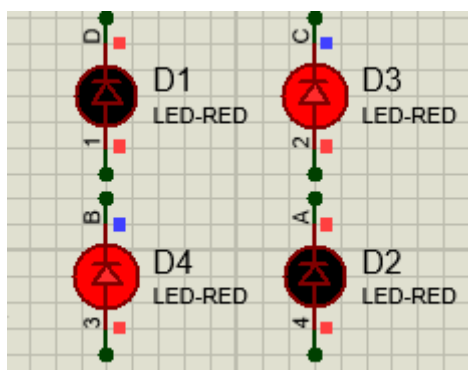


Рисунок 17 – Исходная индикация часов

Нажмите кнопку **INC**, увеличив значение часов на 1. На индикации фиксируется новое значение часов.

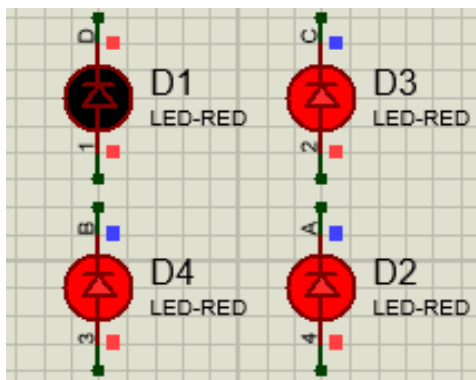


Рисунок 18 – Индикация часов после нажатия кнопки INC

Нажмите кнопку **DEC** два раза, уменьшив значение часов на 2. На индикации фиксируется новое значение часов.

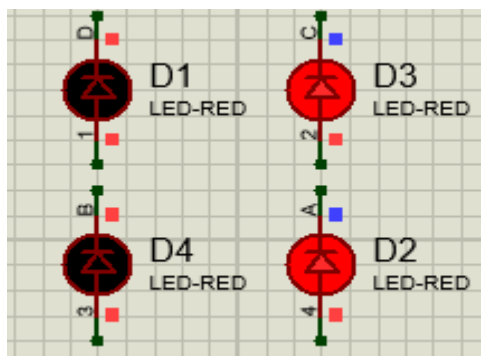


Рисунок 19 – Индикация часов после нажатия кнопки DEC

После установки требуемого значения часов нажмите кнопку **OK** ещё три раза. Таким образом выполняется переход через режимы настройки минут и секунд и осуществляется выход из режима редактирования. После возврата в режим отображения устройство продолжает работу с установленными пользователем параметрами, что подтверждает корректность функции ручной установки времени.

Настройка **минут** и **секунд** выполняется аналогично: соответствующий параметр выбирается последовательным нажатием **OK**, после чего значения изменяются кнопками **INC** и **DEC**. Четвёртое нажатие **OK** завершает режим редактирования, и часы продолжают работу с установленными пользователем параметрами.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано микроконтроллерное устройство «Бинарные наручные часы», предназначенное для отображения текущего времени в двоичном формате с помощью светодиодов. Устройство выполняет индикацию часов, минут и секунд, отображает часть суток с использованием светодиодов АМ/РМ и позволяет вручную устанавливать и корректировать время кнопками ОК/INC/DEC. Точный ход времени и его сохранение при отключении основного питания обеспечиваются применением часов реального времени DS1307 с резервным питанием.

Основные технические характеристики: микроконтроллер AT89C52 (семейство 8051), RTC DS1307 с кварцем 32,768 кГц, программный интерфейс I²C, индикация светодиодов для часов/минут/секунд и 2 светодиода АМ/РМ, три кнопки управления и основное питание. Разработка может использоваться как учебный и демонстрационный проект по микроконтроллерным системам.

Проект размещён в репозитории: <https://github.com/IlyaSevastyanov/binwatch-8051> .

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Tokyoflash. Logo Binary / Logo Wood [Электронный ресурс] // Tokyoflash. – URL: https://tokyoflash.com/products/logo-binary-led-watch?srsltid=AfmBOopzNSrZ7QfHCSlb0gFI_B6mrNFs44N-onAgLcVVSJLhdsQjmi7y7 (дата обращения: 27.09.2025).
2. Tokyoflash. R75 Binary LED Watch [Электронный ресурс] // Tokyoflash. – URL: <https://tokyoflash.com/products/r75-binary-led-watch?srsltid=AfmBOooiGpu6XiEb5THTy3FwstvX2ohJZNSdJGkgZLdbuEpAUazxQ9FD> (дата обращения: 27.09.2025).
3. Tokyoflash Design Studio Blog. Binary LED Watch: Minimal, Sleek and Geeky [Электронный ресурс] // Blog Tokyoflash. – URL: <https://blog.tokyoflash.com/2012/07/30/binary-led-watch-minimal-sleek-and-geeky/> (дата обращения: 27.09.2025).
4. [Обзор Casio] F-91W - протест технологиям, терроризм и новая популярность [Электронный ресурс] // dzen.com. – URL: <https://dzen.ru/a/XrPMD7lefmm9Lvqm?ysclid=mgrym4chqm32024086> (дата обращения: 27.09.2025).
5. Сайт «Паяльник». Двоичные часики на ATmega8 [Электронный ресурс]. – URL: <https://cxem.net/mc/mc247.php> (дата обращения: 27.09.2025).

ПРИЛОЖЕНИЕ А (принципиальная схема)

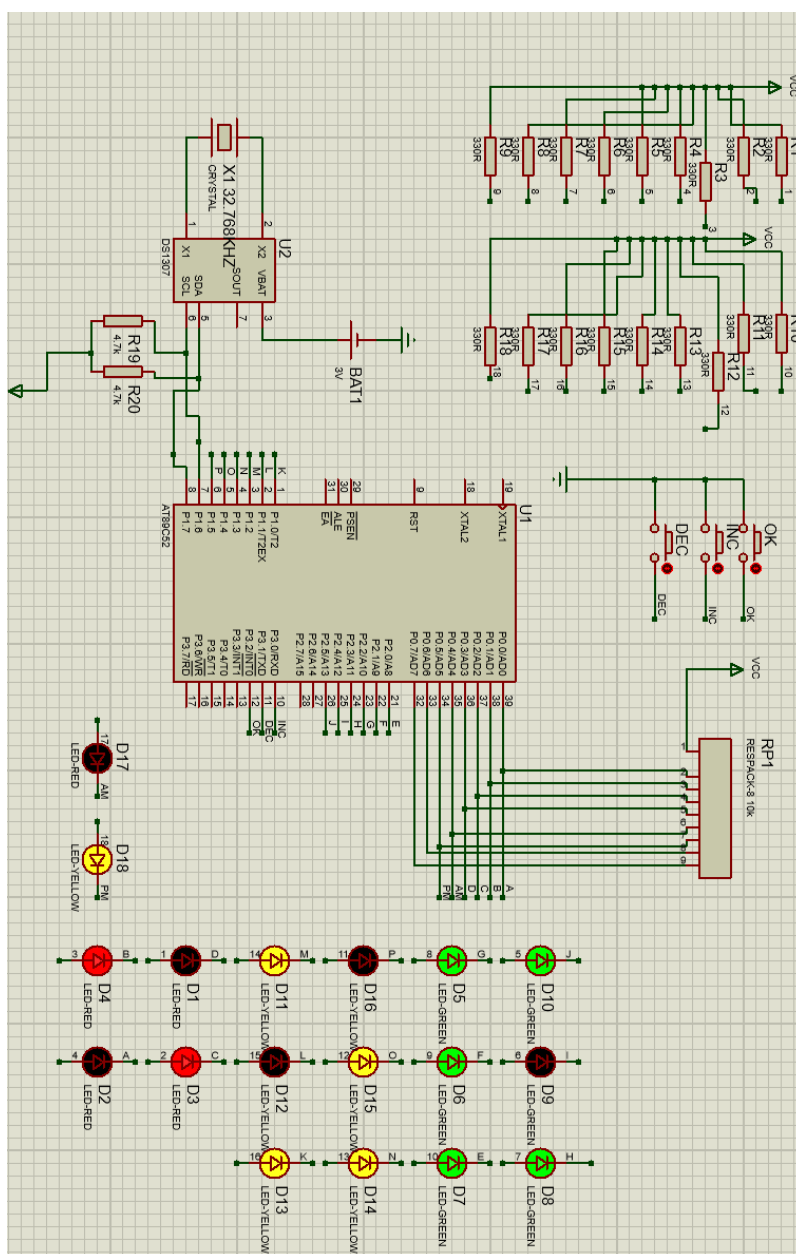


Рисунок А.1 - Принципиальная схема проекта в Proteus

ПРИЛОЖЕНИЕ Б

(текст программы)

```
#include <8051.h> // SDCC: AT89C52 = семейство 8051
// ===== ПИНЫ ИНДИКАЦИИ =====
// P0: часы (4 бита) + AM/PM
__sbit __at(0x80) H0; // P0.0 -> 1
__sbit __at(0x81) H1; // P0.1 -> 2
__sbit __at(0x82) H2; // P0.2 -> 4
__sbit __at(0x83) H3; // P0.3 -> 8
__sbit __at(0x84) LED_AM; // P0.4 -> AM
__sbit __at(0x85) LED_PM; // P0.5 -> PM

// P1: минуты (6 бит)
__sbit __at(0x90) M0; // P1.0 -> 1
__sbit __at(0x91) M1; // P1.1 -> 2
__sbit __at(0x92) M2; // P1.2 -> 4
__sbit __at(0x93) M3; // P1.3 -> 8
__sbit __at(0x94) M4; // P1.4 -> 16
__sbit __at(0x95) M5; // P1.5 -> 32

// P2: секунды (6 бит)
__sbit __at(0xA0) S0; // P2.0 -> 1
__sbit __at(0xA1) S1; // P2.1 -> 2
__sbit __at(0xA2) S2; // P2.2 -> 4
__sbit __at(0xA3) S3; // P2.3 -> 8
__sbit __at(0xA4) S4; // P2.4 -> 16
__sbit __at(0xA5) S5; // P2.5 -> 32

// Кнопки (активный 0)
__sbit __at(0xB0) BTN_INC; // P3.0
__sbit __at(0xB1) BTN_DEC; // P3.1
__sbit __at(0xB2) BTN_OK; // P3.2

// ===== I2C на P1.6/P1.7 =====
__sbit __at(0x96) I2C_SCL; // P1.6
__sbit __at(0x97) I2C_SDA; // P1.7

static void i2c_delay(void) { // ~2-3 мкс при 12 МГц
    unsigned char i; for (i = 0; i < 15; i++) { __asm nop __endasm; }
}
static void sda_release(void) { I2C_SDA = 1; }
static void sda_low(void) { I2C_SDA = 0; }
static void scl_release(void) { I2C_SCL = 1; }
static void scl_low(void) { I2C_SCL = 0; }

static void i2c_start(void) {
    sda_release(); scl_release(); i2c_delay();
    sda_low(); i2c_delay();
    scl_low(); i2c_delay();
}
```

```

static void i2c_stop(void) {
    sda_low(); i2c_delay();
    scl_release(); i2c_delay();
    sda_release(); i2c_delay();
}

static unsigned char i2c_write(unsigned char byte) {
    unsigned char i, ack;
    for (i = 0; i < 8; i++) {
        if (byte & 0x80) sda_release(); else sda_low();
        i2c_delay();
        scl_release(); i2c_delay();
        scl_low(); i2c_delay();
        byte <<= 1;
    }
    // ACK
    sda_release(); i2c_delay();
    scl_release(); i2c_delay();
    ack = !I2C_SDA;
    scl_low(); i2c_delay();
    return ack;
}

static unsigned char i2c_read(unsigned char ack) {
    unsigned char i, data = 0;
    sda_release();
    for (i = 0; i < 8; i++) {
        data <<= 1;
        scl_release(); i2c_delay();
        if (I2C_SDA) data |= 1;
        scl_low(); i2c_delay();
    }
    // отправить ACK/NACK
    if (ack) sda_low(); else sda_release();
    i2c_delay();
    scl_release(); i2c_delay();
    scl_low(); i2c_delay();
    sda_release();
    return data;
}

// ===== DS1307 =====
#define DS1307_ADDR_W 0xD0
#define DS1307_ADDR_R 0xD1

static unsigned char bcd2bin(unsigned char x) { return (x & 0x0F) + 10 * (x >> 4); }
static unsigned char bin2bcd(unsigned char x) { return (x % 10) | ((x / 10) << 4); }

static unsigned char ds1307_write_reg(unsigned char reg, unsigned char val) {
    i2c_start();
    if (!i2c_write(DS1307_ADDR_W)) { i2c_stop(); return 0; }
    if (!i2c_write(reg)) { i2c_stop(); return 0; }
    if (!i2c_write(val)) { i2c_stop(); return 0; }
    i2c_stop();
}

```

```

    return 1;
}
static unsigned char ds1307_read_reg(unsigned char reg, unsigned char* val) {
    i2c_start();
    if (!i2c_write(DS1307_ADDR_W)) { i2c_stop(); return 0; }
    if (!i2c_write(reg)) { i2c_stop(); return 0; }
    i2c_start();
    if (!i2c_write(DS1307_ADDR_R)) { i2c_stop(); return 0; }
    *val = i2c_read(0); // NACK
    i2c_stop();
    return 1;
}

// Запуск осциллятора: очистить бит CH (bit7 в секундах)
static void ds1307_start_osc_if_needed(void) {
    unsigned char sec;
    if (!ds1307_read_reg(0x00, &sec)) return;
    if (sec & 0x80) {
        sec &= 0x7F;
        ds1307_write_reg(0x00, sec);
    }
}

// Чтение H:M:S в 12-часовом формате
// hh: 1..12, pm: 0/1
static unsigned char ds1307_read_time(unsigned char* hh, unsigned char* mm, unsigned char*
ss, unsigned char* pm) {
    unsigned char rs, rm, rh;

    if (!ds1307_read_reg(0x00, &rs)) return 0;
    if (!ds1307_read_reg(0x01, &rm)) return 0;
    if (!ds1307_read_reg(0x02, &rh)) return 0;

    rs &= 0x7F; // CH=0
    *ss = bcd2bin(rs);
    *mm = bcd2bin(rm & 0x7F);

    // нормализация минут/секунд
    if (*ss > 59) *ss = 0;
    if (*mm > 59) *mm = 0;

    if (rh & 0x40) {
        // 12h формат DS1307
        *pm = (rh & 0x20) ? 1 : 0;
        *hh = bcd2bin(rh & 0x1F); // 1..12
        if (*hh < 1 || *hh > 12) *hh = 12;
    }
    else {
        // 24h формат DS1307 -> представление 12h без операций %
        unsigned char h24 = bcd2bin(rh & 0x3F);
        if (h24 > 23) h24 = 0;
    }
}

```



```

    if (h24 == 0) {
        *hh = 12; *pm = 0;
    }
    else if (h24 < 12) {
        *hh = h24; *pm = 0;
    }
    else if (h24 == 12) {
        *hh = 12; *pm = 1;
    }
    else {
        *hh = (unsigned char)(h24 - 12); *pm = 1;
    }
}
return 1;
}

// Запись H:M:S в 12-часовом формате, CH=0
// hh: 1..12, pm: 0/1
static unsigned char ds1307_write_time(unsigned char hh, unsigned char mm, unsigned char ss,
unsigned char pm) {
    unsigned char rs = bin2bcd(ss) & 0x7F; // CH=0
    unsigned char rm = bin2bcd(mm) & 0x7F;

    // 12h: bit6=1, bit5=PM, bits4..0 = BCD(1..12)
    unsigned char rh = bin2bcd(hh) & 0x1F;
    rh |= 0x40;
    if (pm) rh |= 0x20;

    i2c_start();
    if (!i2c_write(DS1307_ADDR_W)) { i2c_stop(); return 0; }
    if (!i2c_write(0x00)) { i2c_stop(); return 0; }
    if (!i2c_write(rs)) { i2c_stop(); return 0; }
    if (!i2c_write(rm)) { i2c_stop(); return 0; }
    if (!i2c_write(rh)) { i2c_stop(); return 0; }
    i2c_stop();
    return 1;
}

// ===== ЛОГИКА ЧАСОВ =====
volatile unsigned char hh = 1, mm = 58, ss = 0; // локальные копии (12h)
volatile unsigned char is_pm = 0; // 0=AM, 1=PM
volatile __bit have_rtc = 0;

typedef enum { ST_SHOW = 0, ST_SET_H, ST_SET_M, ST_SET_S } state_t;
volatile state_t ui = ST_SHOW;

volatile unsigned int ms = 0;
volatile __bit tick_1s = 0;

// состояния кнопок (без дебаунса)
volatile __bit prev_inc = 0, prev_dec = 0, prev_ok = 0;

```

```

// ===== Вспомогательная логика 12h =====
static void hour_inc_12(void) {
    if (hh == 11) {
        hh = 12;
        is_pm = !is_pm;
    }
    else if (hh == 12) {
        hh = 1;
    }
    else {
        hh++;
    }
}

static void hour_dec_12(void) {
    if (hh == 12) {
        hh = 11;
        is_pm = !is_pm;
    }
    else if (hh == 1) {
        hh = 12;
    }
    else {
        hh--;
    }
}

// ===== Индикация (катоды на МК: 0=ВКЛ, 1=ВЫКЛ) =====
static void set_hour_leds(unsigned char v4) {
    H0 = (v4 & 0x01) ? 0 : 1;
    H1 = (v4 & 0x02) ? 0 : 1;
    H2 = (v4 & 0x04) ? 0 : 1;
    H3 = (v4 & 0x08) ? 0 : 1;
}

static void set_min_leds(unsigned char v6) {
    M0 = (v6 & 0x01) ? 0 : 1;
    M1 = (v6 & 0x02) ? 0 : 1;
    M2 = (v6 & 0x04) ? 0 : 1;
    M3 = (v6 & 0x08) ? 0 : 1;
    M4 = (v6 & 0x10) ? 0 : 1;
    M5 = (v6 & 0x20) ? 0 : 1;
}

static void set_sec_leds(unsigned char v6) {
    S0 = (v6 & 0x01) ? 0 : 1;
    S1 = (v6 & 0x02) ? 0 : 1;
    S2 = (v6 & 0x04) ? 0 : 1;
    S3 = (v6 & 0x08) ? 0 : 1;
    S4 = (v6 & 0x10) ? 0 : 1;
    S5 = (v6 & 0x20) ? 0 : 1;
}

static void set_ampm(void) {
    LED_AM = (is_pm == 0) ? 0 : 1;
    LED_PM = (is_pm == 1) ? 0 : 1;
}

```

```

}
static void refresh_display(void) {
    unsigned char hdisp = hh & 0x0F; // hh уже 1..12
    set_hour_leds(hdisp);
    set_min_leds(mm);
    set_sec_leds(ss);
    set_ampm();
}

// ===== Таймер0: 1 мс тик =====
void timer0_isr(void) __interrupt(1) {
    TH0 = 0xFC; TL0 = 0x18;
    if (++ms >= 1000) { ms = 0; tick_1s = 1; }
}

// ===== Обработка кнопок (edge, без дебаунса) =====
static void apply_inc(void) {
    if (ui == ST_SET_H) {
        hour_inc_12();
    }
    else if (ui == ST_SET_M) {
        if (mm < 59) mm++;
        else mm = 0;
    }
    else if (ui == ST_SET_S) {
        if (ss < 59) ss++;
        else ss = 0;
    }
    refresh_display();
}

static void apply_dec(void) {
    if (ui == ST_SET_H) {
        hour_dec_12();
    }
    else if (ui == ST_SET_M) {
        if (mm > 0) mm--;
        else mm = 59;
    }
    else if (ui == ST_SET_S) {
        if (ss > 0) ss--;
        else ss = 59;
    }
    refresh_display();
}

static void process_keys(void) {
    __bit inc_now = !BTN_INC;
    __bit dec_now = !BTN_DEC;
    __bit ok_now = !BTN_OK;

    // короткое ОК: по отпусканию

```

```

    if (!ok_now && prev_ok) {
        if (ui == ST_SHOW) ui = ST_SET_H;
        else if (ui == ST_SET_H) ui = ST_SET_M;
        else if (ui == ST_SET_M) ui = ST_SET_S;
        else ui = ST_SHOW;
    }

    if (inc_now && !prev_inc) apply_inc();
    if (dec_now && !prev_dec) apply_dec();

    prev_inc = inc_now; prev_dec = dec_now; prev_ok = ok_now;
}

// ===== Инициализация =====
static void timer0_init(void) {
    TMOD = (TMOD & ~0x0F) | 0x01; // T0 mode1
    TH0 = 0xFC; TL0 = 0x18; ET0 = 1; TR0 = 1;
}

void main(void) {
    P0 = 0xFF; P1 = 0xFF; P2 = 0xFF; P3 = 0xFF;

    I2C_SCL = 1; I2C_SDA = 1;

    timer0_init(); EA = 1;

    ds1307_start_osc_if_needed();

    {
        unsigned char rh, rm, rs, rpm;
        if (ds1307_read_time(&rh, &rm, &rs, &rpm)) {
            hh = rh; mm = rm; ss = rs; is_pm = rpm;
            have_rtc = 1;
        }
        else {
            have_rtc = 0;
        }
    }

    refresh_display();

    state_t prev_ui = ui;

    prev_inc = !BTN_INC; prev_dec = !BTN_DEC; prev_ok = !BTN_OK;

    for (;;) {
        if (tick_1s) {
            tick_1s = 0;

            if (ui == ST_SHOW) {
                if (have_rtc) {
                    unsigned char rh, rm, rs, rpm;

```

```

        if (ds1307_read_time(&rh, &rm, &rs, &rpm)) {
            hh = rh; mm = rm; ss = rs; is_pm = rpm;
        }
    }
    else {
        // внутренний ход времени
        if (++ss >= 60) {
            ss = 0;
            if (++mm >= 60) {
                mm = 0;
                hour_inc_12();
            }
        }
    }
    refresh_display();
}

process_keys();

// Если выходим из режима SET_* в SHOW — записываем в RTC
if (prev_ui != ui && ui == ST_SHOW && have_rtc) {
    ds1307_write_time(hh, mm, ss, is_pm);
}
prev_ui = ui;
}
}

```